Advanced Network Analyzer

Comprehensive Project Documentation

Subject: Computer Networks

Technologies: Python, Scapy, Streamlit, Plotly, Pandas

Real-Time Packet Capture | Interactive Dashboard | ARP Spoofing Detection

Table of Contents

1. Introduction and Project Overview
2. Project Structure and File Organization
3. System Architecture
4. OSI / TCP-IP Layer Mapping
5. Detailed Component Breakdown
6. Protocol Detection Mechanism
7. Security Module: ARP Spoofing Detection
8. End-to-End Data Flow
9. Technologies Used
10. Educational Outcomes and Conclusion

1. Introduction and Project Overview

What is the Advanced Network Analyzer?

The Advanced Network Analyzer is a full-stack, real-time network monitoring application built entirely in Python. It combines low-level packet sniffing with a modern, interactive web-based dashboard to provide comprehensive visibility into local network traffic.

In production and enterprise environments, network administrators use heavyweight tools like Wireshark or tcpdump for packet-level analysis. This project offers a lightweight, extensible, and visually appealing alternative that is ideal for educational environments and demonstrations.

What Does the Project Do?

The application performs the following core functions:

- Captures live network packets in real-time from the host machine's network interface (Wi-Fi or Ethernet).
- Classifies each packet by identifying its protocol at both the transport layer (TCP, UDP, ICMP) and the application layer (HTTP, HTTPS, DNS, SSH, FTP, DHCP, NTP, SNMP, etc.) using port-number mapping.
- Extracts DNS queries to show exactly which domain names the machine is resolving (e.g., www.google.com, api.github.com).
- Detects ARP Spoofing attacks on the local network by maintaining a stateful IP-to-MAC address mapping table and checking for anomalies.
- Visualizes all captured data through an interactive Streamlit web dashboard with Plotly charts, including protocol distribution pie charts, bar charts, packet size histograms, and traffic volume analysis.
- Provides real-time monitoring with auto-refreshing views and the ability to start/stop captures directly from the browser-based UI.

Why This Project Matters for Computer Networks

This project directly demonstrates the following fundamental Computer Networks concepts: the TCP/IP protocol stack, packet encapsulation and decapsulation across OSI layers, IP addressing and routing, the role of transport-layer ports in multiplexing, the DNS resolution process, ARP protocol mechanics and its vulnerabilities, and network traffic analysis techniques. By building this system, one gains practical, hands-on experience with concepts that are typically only studied theoretically in textbooks.

2. Project Structure and File Organization

The project follows a clean, modular directory structure that separates concerns between the backend engine and the frontend UI:

```
Advanced Network Analyzer/
|
|-- core/
|   |-- sniffer.py          # Backend packet capture engine
|
|-- data/
|   |-- traffic_data.json   # Live packet data (auto-generated)
|   |-- status.json         # Capture status (auto-generated)
|
|-- progress/               # Build context and tracking files
|
|-- app.py                  # Streamlit frontend dashboard
|-- requirements.txt        # Python dependencies
|-- generate_pdf.py         # This PDF generator script
```

File Responsibilities:

- core/sniffer.py: The heart of the application. Contains the NetworkSniffer class that uses Scapy to capture packets, a Packet Processor that classifies protocols, an ARP Spoofing Detector, and a background thread that serializes data to disk.
- app.py: The Streamlit web application that serves as the visual control center. It reads the JSON data files, renders interactive Plotly visualizations, and provides controls to start/stop the capture engine.
- data/traffic_data.json: The shared data bridge between backend and frontend. Contains the list of captured packets, protocol distribution counts, DNS query logs, and ARP spoofing alerts.
- data/status.json: A lightweight file indicating whether the capture engine is currently running and how many packets have been captured so far.
- requirements.txt: Lists the four core Python dependencies: scapy, pandas, streamlit, and plotly.

3. System Architecture

The system is designed as a decoupled, two-process architecture. The backend capture engine and the frontend dashboard run as completely separate processes that communicate through a shared file-based data bridge. This decoupling provides several important benefits:

- Reliability: If the UI crashes or is restarted, the backend continues capturing packets without interruption.
- Performance: Heavy packet processing does not block UI rendering, and UI interactions (filtering, scrolling) do not affect capture performance.
- Simplicity: Using JSON files as the communication layer avoids the complexity of databases, message queues, or socket-based IPC.

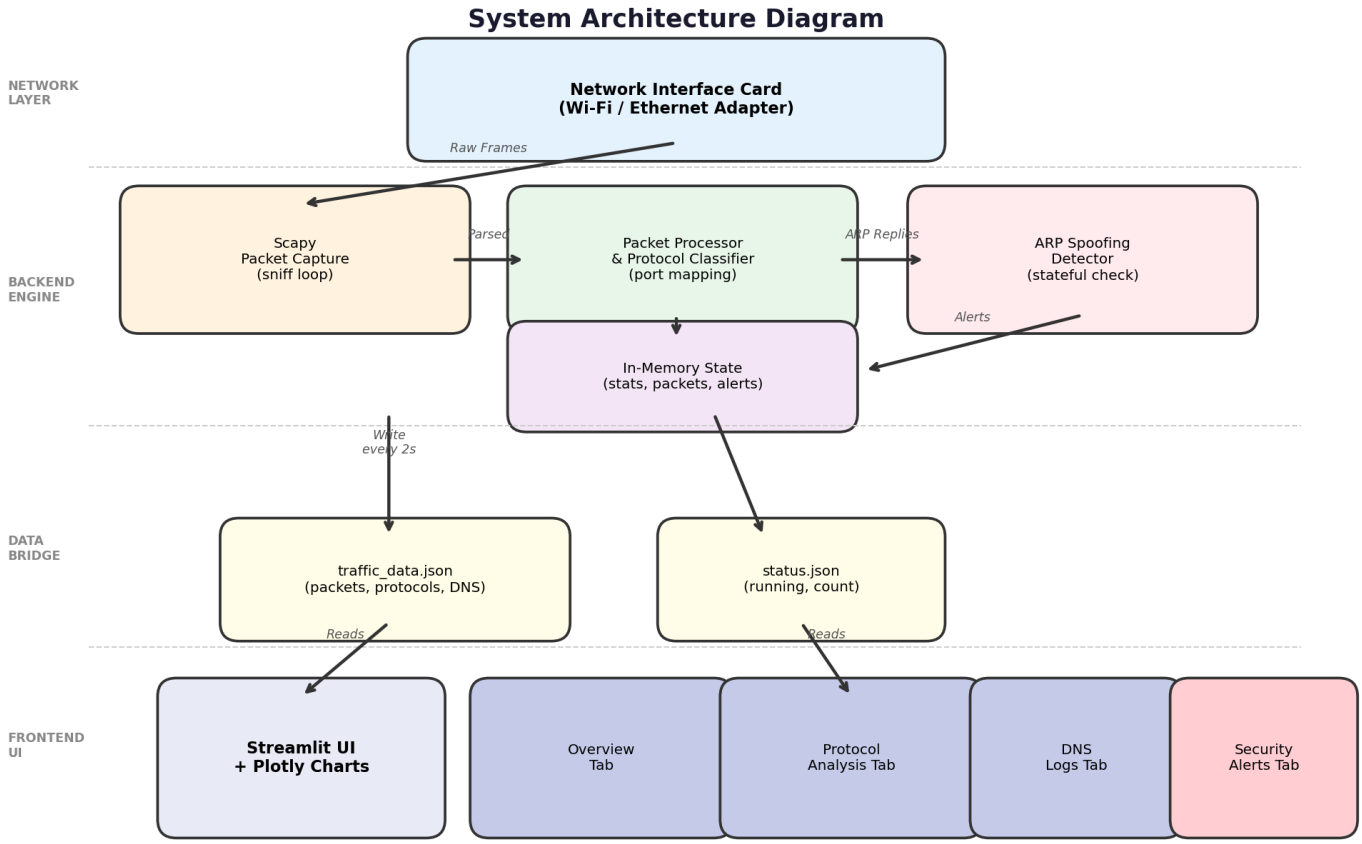


Figure 1: Complete System Architecture Diagram showing all components and data flow

As shown in Figure 1, the architecture is organized into four layers: (1) The Network Layer provides raw Ethernet frames from the NIC. (2) The Backend Engine processes these frames through Scapy, classifies protocols, and checks for ARP anomalies. (3) The Data Bridge (JSON files) acts as the asynchronous communication channel. (4) The Frontend UI reads this data and renders it through interactive Plotly charts organized across four specialized tabs.

4. OSI / TCP-IP Layer Mapping

One of the key educational aspects of this project is how it interacts with nearly every layer of the OSI model. The diagram below maps each OSI layer to the specific data that the analyzer extracts and processes at that layer.

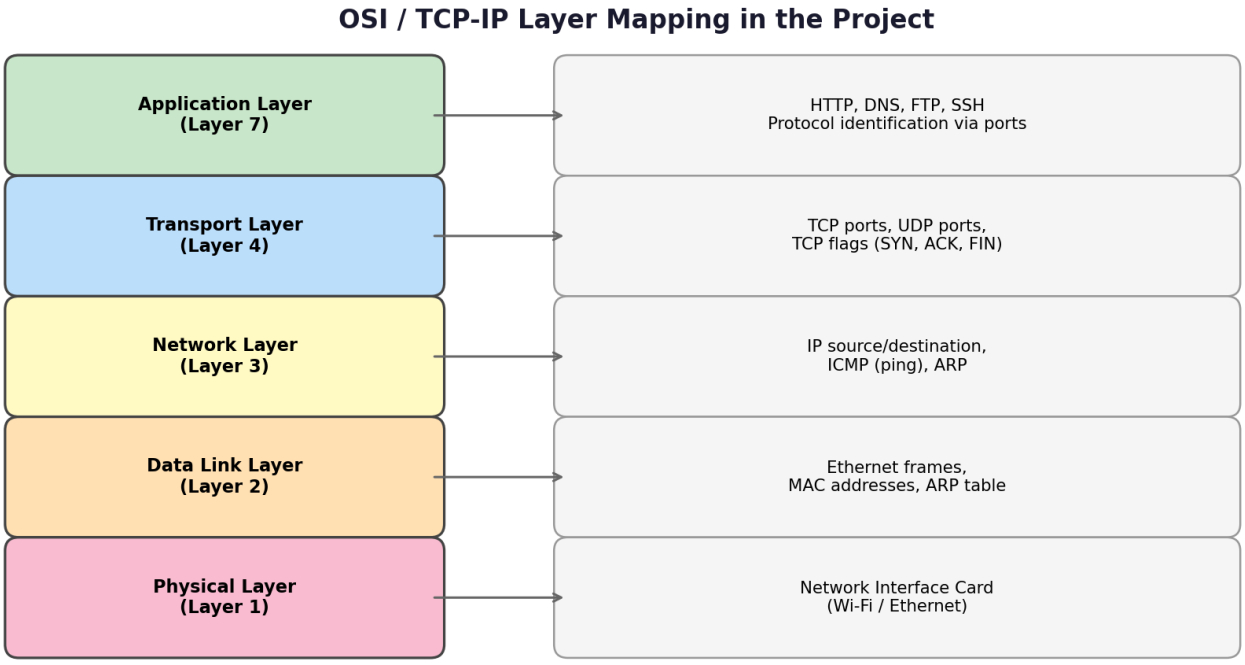


Figure 2: How the project maps to each layer of the OSI / TCP-IP model

Physical Layer (Layer 1): The project relies on the host's Network Interface Card (NIC) to convert electrical or radio signals into digital frames. Scapy binds to this interface to receive raw data.

Data Link Layer (Layer 2): Ethernet frames contain source and destination MAC addresses. The ARP protocol operates at this layer. Our ARP spoofing detector maintains an IP-to-MAC mapping table and monitors ARP replies for inconsistencies.

Network Layer (Layer 3): IP packets carry source and destination IP addresses. The project extracts these addresses for traffic analysis, showing top source IPs and destination IPs. ICMP (ping) also operates at this layer and is captured and displayed.

Transport Layer (Layer 4): TCP and UDP segments carry port numbers that enable multiplexing. The project uses these port numbers as the primary mechanism for identifying application-layer protocols (e.g., port 80 = HTTP, port 443 = HTTPS).

Application Layer (Layer 7): For specific protocols like DNS, the project performs deep packet inspection. It reads the DNS Question Record (DNSQR) inside UDP port 53 packets to extract the actual domain name being queried.

5. Detailed Component Breakdown

5.1 Backend Capture Engine (core/sniffer.py)

The NetworkSniffer class is the core of the application. When instantiated and started, it launches two daemon threads: a capture thread and a save thread.

The Capture Thread calls Scapy's sniff() function in a continuous loop. Scapy uses Berkeley Packet Filter (BPF) at the kernel level for efficient, low-overhead packet interception. Each captured packet is passed to the _process_packet() callback method, which dissects it layer by layer:

- First, it checks for ARP packets (Layer 2). If found, it extracts the ARP operation (Request or Reply), the source IP, and the destination IP.
- Next, it checks for an IP layer (Layer 3). If absent, the packet is a non-IP frame and is skipped.
- It then checks for TCP (extracting source/destination ports and mapping them to known services), UDP (similarly mapping ports, with special handling for DNS packets), or ICMP (identifying ping requests and replies).
- For DNS packets specifically, it decodes the DNSQR query name to reveal the domain being resolved.

The Save Thread wakes up every 2 seconds and writes the entire in-memory state (packets list, protocol counts, DNS queries, and ARP alerts) to traffic_data.json. It also updates status.json with the current running state and packet count. This periodic flush ensures the frontend always has fresh data without requiring a direct network connection between the processes.

Memory Management: To prevent unbounded memory growth during long capture sessions, the engine maintains a sliding window of the last 5,000 packets and the last 500 DNS queries. Older entries are automatically discarded.

5.2 Frontend Dashboard (app.py)

The Streamlit dashboard is organized into a sidebar (for controls and filters) and a main content area with four tabs. Here is a detailed explanation of each component:

- Sidebar - Process Controls: Contains 'Start Live Capture' and 'Stop Capture' buttons. When Start is clicked, the app uses Python's subprocess.Popen to launch core/sniffer.py as an independent background process. The process reference is stored in Streamlit's session state so it can be terminated later.
- Sidebar - Protocol Filters: A multi-select widget populated dynamically from the captured data. Users can select specific protocols (e.g., only DNS and HTTPS) and the entire dashboard re-renders to show only traffic matching those filters.
- Overview Tab: Shows four KPI metric cards (Total Packets, Total Traffic in bytes, Average Packet Size, Unique IPs) and a scrollable table of the most recent 100 captured packets.
- Protocol Analysis Tab: Renders an interactive Plotly donut chart showing protocol distribution and a bar chart showing packet counts per protocol.
- Application Logs Tab: Displays a reverse-chronological table of all intercepted DNS queries, showing timestamp, source IP, and the queried domain name.

- Security Alerts Tab: If ARP spoofing is detected, shows a red alert banner with the count of attacks and a detailed table listing each alert with timestamp, IP, original MAC, and spoofed MAC.

6. Protocol Detection Mechanism

One of the distinguishing features of this project is its ability to identify application-layer protocols rather than simply reporting 'TCP' or 'UDP'. This is achieved through a port-number mapping system. The flowchart below illustrates the decision tree used to classify each packet.

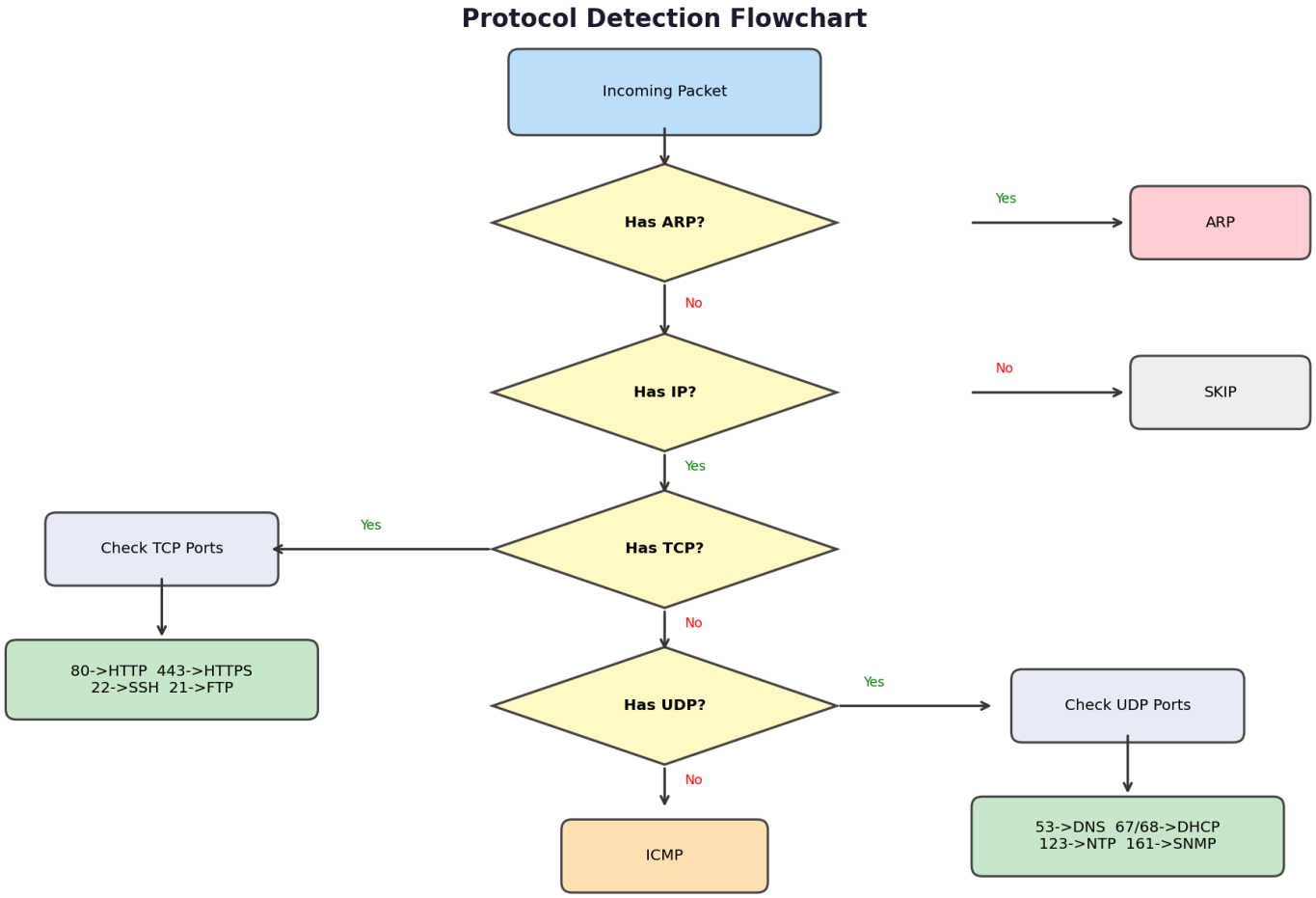


Figure 3: Protocol Detection Flowchart showing the classification decision tree

The classifier works as follows: First, the packet is checked for an ARP layer. If present, it is immediately classified as ARP. Otherwise, the IP layer is checked. If no IP layer exists, the packet is skipped. For IP packets, the transport layer determines the next step.

For TCP packets, the source and destination ports are checked against a dictionary of 18+ known services. For example: Port 80 maps to HTTP, Port 443 to HTTPS, Port 22 to SSH, Port 21 to FTP, Port 3306 to MySQL, Port 3389 to RDP, and Port 8080 to HTTP-ALT. If no known port is matched, the packet is labeled simply as 'TCP'.

For UDP packets, a similar dictionary is used: Port 53 maps to DNS, Ports 67/68 to DHCP, Port 123 to NTP, and Port 161 to SNMP. Additionally, if a UDP packet on port 53 contains a DNS Question Record (DNSQR), the analyzer performs deep packet inspection to decode the queried domain name.

7. Security Module: ARP Spoofing Detection

What is ARP Spoofing?

Address Resolution Protocol (ARP) is used on local area networks to map IP addresses to physical MAC addresses. When a device wants to communicate with another device on the same network, it sends an ARP Request asking 'Who has IP X.X.X.X?'. The device with that IP responds with an ARP Reply containing its MAC address. The requesting device caches this mapping in its ARP table.

ARP is a stateless, trust-based protocol. It accepts ARP Replies even if it never sent a corresponding Request. An attacker can exploit this by sending unsolicited ARP Replies to a victim, claiming that the attacker's MAC address belongs to the router's IP. This redirects all of the victim's internet traffic through the attacker's machine, enabling a Man-In-The-Middle (MITM) attack.

How the Analyzer Detects ARP Spoofing

The detection mechanism is based on stateful inspection of ARP Reply packets:

- Step 1 - State Building: When an ARP Reply is first seen for a given IP, the analyzer stores the IP-to-MAC mapping in an internal dictionary (arp_table).
- Step 2 - Continuous Monitoring: For every subsequent ARP Reply, the analyzer extracts the claimed IP and MAC address from the packet.
- Step 3 - Anomaly Detection: The extracted MAC is compared against the stored MAC for that IP. If they differ, it means someone is attempting to change the routing path.
- Step 4 - Alert Generation: An alert object is created containing the timestamp, the affected IP, the original (legitimate) MAC, and the new (spoofed) MAC. This alert is pushed to the shared JSON state.
- Step 5 - Visual Notification: The Streamlit frontend reads the alerts and displays them prominently in the Security Alerts tab with a red warning banner.

ARP Spoofing Detection Mechanism

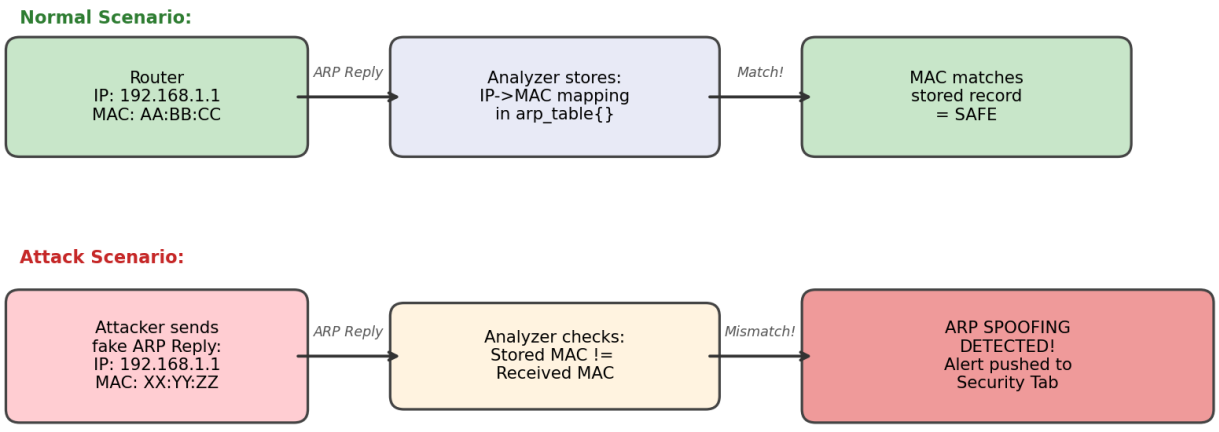


Figure 4: ARP Spoofing Detection - Normal vs Attack scenario comparison

8. End-to-End Data Flow

The following diagram illustrates the complete chronological sequence of events from the moment the user initiates a capture session to the visual rendering of network traffic data on the dashboard.

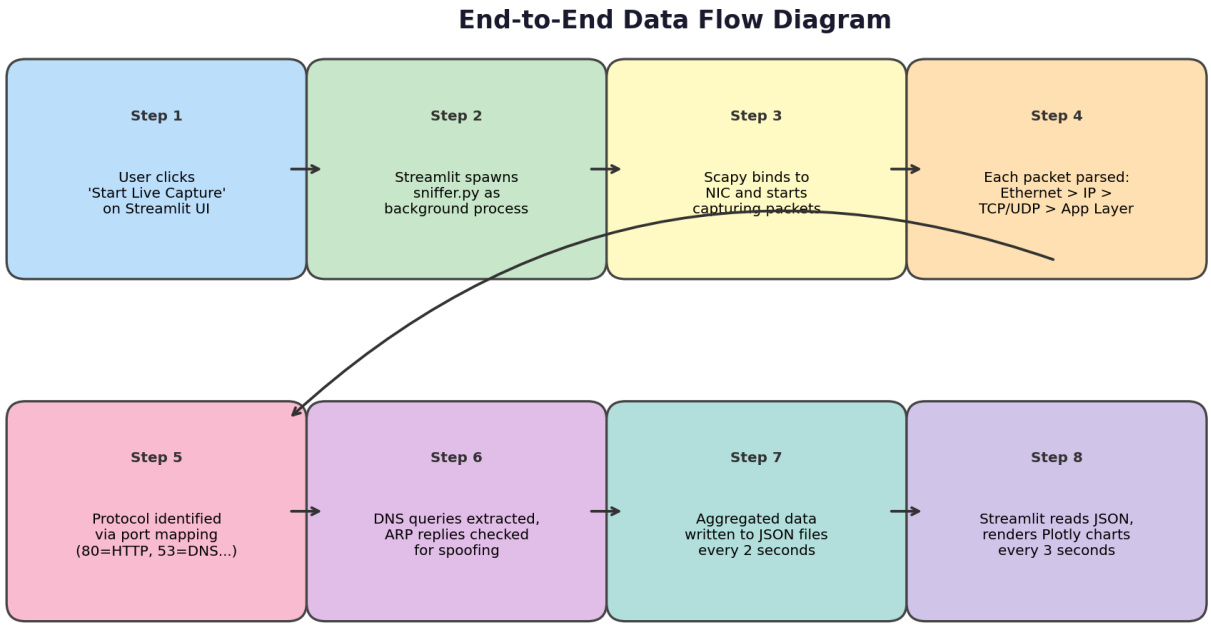


Figure 5: End-to-End Data Flow showing all 8 steps from user action to visualization

Step 1: The user opens the Streamlit dashboard in their browser (typically at http://localhost:8501) and clicks the 'Start Live Capture' button in the sidebar.

Step 2: The Streamlit application executes subprocess.Popen() to spawn core/sniffer.py as an independent background process. This ensures the capture runs independently of the UI.

Step 3: The sniffer initializes Scapy and binds to the default network interface. Scapy configures a raw socket to receive copies of all packets flowing through the NIC.

Step 4: As each packet arrives, it is dissected layer by layer - from Ethernet frame to IP header to TCP/UDP segment to application-layer payload.

Step 5: Port numbers are mapped to known services (HTTP, DNS, SSH, etc.). Special handling extracts DNS query names and checks ARP packets against the stored state.

Step 6: The DNS queries extracted from the packets are appended to a list. ARP Reply packets are cross-referenced with the arp_table dictionary to detect any MAC address changes.

Step 7: Every 2 seconds, a dedicated save-thread writes the complete aggregated state (packets, protocol counts, DNS logs, and security alerts) to traffic_data.json on disk.

Step 8: The Streamlit UI is on a 3-second auto-refresh cycle. Each cycle, it reads the updated JSON file, loads it into a

Pandas DataFrame, applies any active protocol filters, and re-renders all Plotly charts and data tables.

9. Technologies Used

The project uses the following technologies, all of which are free and open-source:

Technology	Purpose and Role in the Project
Python 3	Core programming language for both backend and frontend.
Scapy	Packet capture and deep packet inspection library. Uses raw sockets to sniff live traffic from the NIC.
Streamlit	Reactive web framework for building the interactive dashboard. Handles UI rendering and process manageme
Plotly	Interactive charting library for pie charts, bar charts, and histograms displayed on the dashboard.
Pandas	Data manipulation library. Converts JSON data into DataFrames for filtering, aggregation, and display.
JSON (stdlib)	Serialization format for the data bridge between backend and frontend processes.
Threading (stdlib)	Python's threading module runs the capture loop and save loop concurrently in daemon threads.
Subprocess (stdlib)	Launches and manages the backend sniffer as an independent child process from the Streamlit UI.

10. Educational Outcomes and Conclusion

By building and studying this project, one gains deep, practical insights into:

- Packet Structure: Understanding Ethernet frames, IP headers, TCP/UDP segments, and application-layer payloads through hands-on dissection.
- Protocol Identification: Learning how port numbers enable multiplexing and how applications are identified at the transport layer.
- DNS Resolution: Observing the actual DNS queries generated by a machine and understanding the role of recursive resolvers.
- Network Security: Understanding how ARP's trust-based design creates vulnerabilities and how stateful inspection can detect attacks.
- Asynchronous Systems: Learning how multi-threaded Python programs coordinate work using daemon threads and shared state.
- Inter-Process Communication: Understanding how two separate processes (sniffer and UI) can communicate through a decoupled file-based data bridge.
- Data Visualization: Using Plotly and Streamlit to transform raw network data into actionable, human-readable insights.

The Advanced Network Analyzer successfully transforms an abstract, high-speed stream of binary network data into structured, visual, and actionable insights. It serves as both a powerful learning tool for understanding network protocols at every layer of the OSI model, and a practical utility for monitoring local network health and security. The entire system operates 100% locally with zero cloud dependencies, demonstrating that powerful network analysis can be achieved efficiently with Python and free, open-source libraries.